

MySQL底层原理

一、索引篇

第一幕：“目录”的诞生

痛点：数据库的数据是存在磁盘上的。如果我们要找 `id = 500` 的记录，最原始的办法就是从第一行开始挨个对比。这叫**全表扫描**。由于磁盘 I/O 极慢，表里有几千万数据时，这就相当于在大海捞针，系统直接卡死。

解法：为了不挨个找，我们需要像书本一样，给数据建一个“目录”。这个目录就是**索引**。

第二幕：B+ 树的上位

如果用 **Hash 表**，找单个值极快，但没法做范围查询（比如找 `id > 100` 的人）

如果用普通的 **二叉树**，数据多了树会变得特别高，每往下走一层就是一次磁盘 I/O，太慢。

如果用 **B 树**，虽然树变矮了，但每个节点里既有索引又有真实数据，一个 16KB 的页塞不了多少指针，树还是偏高。

解法：于是，InnoDB 选择了 **B+ 树**。实现了“干湿分离”：非叶子节点只存目录（极大地增加了分叉数，让树变得极其矮胖），所有的真实数据全放在最底层的叶子节点，并且用双向链表连起来，完美支持范围查询。

第三幕：主键索引的落地

现状：现在我们有了一棵极其高效的 B+ 树，但真实的数据存在哪呢？

解法：InnoDB 决定“把数据和目录绑在一起”。它默认用**主键**来构建这棵 B+ 树，然后把**完整的一行行真实数据**，直接塞进这棵树的叶子节点里。这棵以主键为核心、叶子节点挂着所有真实数据的 B+ 树，就叫**聚簇索引**

第四幕：二级索引的出现

痛点：聚簇索引是用“主键”排好序的，如果我执行 `SELECT * FROM user WHERE name = '张三'` 怎么办？聚簇索引帮不上忙，难道又要去全表扫描？

解法：为了能按名字查，我们只能再建一棵 B+ 树！这棵树按照 `name` 的字母顺序来排序。

新痛点：那这棵新树的叶子节点里，也要把张三的完整数据（年龄、性别、地址等）再存一份吗？如果按年龄、按性别都建树，数据岂不是要复制无数份？太浪费磁盘了！

解法：InnoDB 非常聪明，它在这棵新树的叶子节点里，**只存 `name` 的值，以及张三对应的“主键 ID”**。这棵只存了部分数据和主键 ID 的辅助树，就叫**二级索引 / 非聚簇索引**

第五幕：二级索引的代价——回表

痛点：现在通过二级索引，极速找到了 `name = '张三'`，并在叶子节点拿到了他的主键 `id = 500`。但是，你的 SQL 语句是 `SELECT *`，也就是要看张三的所有信息！而二级索引里只有名字和 ID，没有地址和年龄啊。

解法：没办法，既然拿到了 `id = 500`，那就带着这个 ID，**回到聚簇索引（主键那棵大树）里，再按主键查一次**，从而拿到完整的这一行数据。这个“从二级索引查到 ID，再回到聚簇索引查完整数据”的过程，就叫**回表（Bookmark Lookup）**。

第六幕：为了干掉回表的优化

痛点：回表意味着要查两棵树，也就是双倍的磁盘 I/O。如果能不回表就好了！

解法 1：覆盖索引（Covering Index） 如果你只是想查张三的 ID，你的 SQL 写成 `SELECT id, name FROM user WHERE name = '张三'`。此时，二级索引的叶子节点里正好有 `id` 和 `name`，你想要的东西这棵树里全都有！那就没必要再去聚簇索引查了，直接返回结果。这就叫**覆盖索引**（索引覆盖了你的查询需求）。

解法 2：联合索引（Composite Index） 如果你经常需要同时按“名字”和“年龄”查人：`WHERE name = '张三' AND age = 20`。你不需要建两棵树，你可以把这两个字段捆绑在一起，建一棵**联合索引**树。这棵树的排序规则是：先按 `name` 排序；如果 `name` 相同，再按 `age` 排序

二、事务篇

第一幕：原子性

痛点：假设你要给朋友转账 100 块。数据库需要做两步：1. 你的账户扣 100；2. 朋友的账户加 100。如果第一步刚做完，服务器机房突然断电了！你的钱扣了，朋友没收到，这钱直接人间蒸发了

解法：就是撤销这个操作。回滚日志（undo log）顺着日志把数据反向修改回去。实现要么成功，要么不做

第二幕：持久性

痛点：现在转账成功了，数据库告诉你 `COMMIT` 完成。但你要知道，为了速度，数据库是把数据先写在**内存里的。如果这个时候服务器拔掉网线，内存里的数据瞬间消失，你刚才成功的转账又没了

解法：写入磁盘，而且增加了redo log 和 WAL技术，按顺序写入的“本子”，重启时只要照着小本子重做一遍，数据就回来了，注意，是先写日志再写盘

第三幕：隔离性

比如脏读，不可重复读，幻读，锁机制实现。

InnoDB 给表中的**每一行数据**都维护了多个“历史版本快照”，不同的事务在访问同一行数据时，会根据规则看到不同版本的快照，而非直接竞争同一份数据。

它的核心目标是：**读操作不加锁，读写操作不冲突**（对比传统锁机制“要么等锁、要么阻塞”的方式），以此大幅提升数据库的并发读写性能。

第四幕：一致性

以上共同完成

三、锁篇

先看自己的总结

首先，为了防止多并发下的错读现象，就出现了行锁，为了防止幻读等，行锁里面就有间隙锁和临建锁
如果需要对整张表进行操作，为了防止冲突，又出现了表锁，之所以有了表锁，又出现行锁，是因为行锁的颗粒度更细，更便于精准操作。
但是，当事务A锁了某一行时，如果事务B想锁这一张表，行锁和表锁就会产生冲突，于是事务B就会先查询是否有锁，但是一行一行查询又太慢，于是就有了意向锁

第一：关于行锁与幻读

最基础的行锁（记录锁）是为了保证大家并发修改不同行时不打架。但如果你执行的是范围查询，别人悄悄在你的范围里“塞”了一行新数据（幻读），基础的行锁就防不住了，因为你没法锁住原本不存在的东西。为了堵住这个漏洞，InnoDB 只能把记录之间的空隙也锁上，这就是间隙锁（Gap Lock）和临键锁（Next-Key Lock）诞生的必然结果。

第二：颗粒度的权衡

表锁是“一刀切”，加锁极快、极度安全，但并发性能极差，大家都要排队；行锁是“精雕细琢”，并发性能极高，但开销大、实现极其复杂。InnoDB 引入行锁，就是为了在保障安全的前提下，榨干数据库的最大并发吞吐量。

第三：意向锁

如果没有意向锁，事务 B 想要给整张表加锁时，它面临的情况是崩溃的：它必须从头到尾遍历整棵 B+ 树，检查几千万条记录中是不是有哪怕一条记录被事务 A 锁住了。这种检测效率低得令人发指。

而意向锁巧妙地解决了这个问题：事务 A 在给某一行加锁前，必须先在表级别挂上一个意向锁（相当于在酒店大堂挂了个牌子：“本酒店有房间正在被使用”）。事务 B 想锁整张表时，连 B+ 树都不用进，直接抬头看一眼表上有没有意向锁，有的话就立刻进入等待状态。完美把冲突检测的代价从遍历整表降到了一步到位

四、日志篇

第一：撤销日志和重做日志

撤销日志用于**保证事务原子性**，记录事务执行前的数据旧版本。当事务执行失败、需要回滚，或数据库崩溃时，可通过它将数据恢复到修改前的状态；同时它还支撑多版本并发控制（MVCC），让不同事务能读取到对应版本的数据，实现无锁读写。

重做日志用于**保证事务持久性**，记录事务执行后的修改操作。为提升性能，数据库修改先写内存、再异步落盘，若中途宕机，已提交事务的修改可能丢失；重启时通过重做日志重放所有已提交的修改，确保数据永久生效、不丢失。

第二：binlog的上位

redo log 没有记忆，而且独属于 InnoDB，无法实现主从架构

Binlog: MySQL 在更高的 **Server 层**（所有存储引擎共用）建立了一个全局的账本，它是追加写的，不会覆盖历史记录，而且是逻辑日志，记录了所有增删改语句和行级数据变化，可以把数据库精准恢复到过去的任何一秒钟的状态。同时，主库把 Binlog 发给从库，从库照着日志把 SQL 重新执行一遍，实现数据实时同步，因为所有数据库都认识 binlog

第三：两阶段提交

当执行一条 `UPDATE` 语句时，InnoDB 需要写 Redo Log，Server 层需要写 Binlog。这就有了两套账本。

假设我们先写 Redo Log，再写 Binlog，Redo Log 刚写完，机器断电了，Binlog 没来得及写。

后果：机器重启后，由于 Redo Log 写了，主库的数据被恢复了（扣了 100 块钱）。但是因为 Binlog 没写，从库同步不到这条消息。结果就是：主库扣钱了，从库没扣钱。主从数据永久不一致！

解法：InnoDB 先把数据写到 Redo Log 中，并设置为 **prepare 状态**，server 层将这次操作的逻辑记录到 Binlog 中，当 InnoDB 收到 Binlog 写成功的消息后，再把 redo log 的状态改为 **commit**，事务正式完成。如果 Redo Log 里的状态是 `Prepare`，说明断电发生在中间。此时 MySQL 会去查对应的 Binlog，如果 Binlog 里有这条记录，说明 Binlog 写完了才断电的，那就顺水推舟，把 Redo Log 强行 Commit，如果

Binlog 里**没有**这条记录，说明 Binlog 没写完，为了保证主从一致，直接把 Redo Log 给 回滚

五、内存篇

第一：Buffer Pool的诞生

如果每次select 都要去磁盘，就会导致效率很慢，而且磁盘I/O的时间很长。InnoDB在内存里划出来一块缓冲池，当第一次查数据时，InnoDB 会把包含这条数据的整个 16KB 数据页从磁盘加载到 Buffer Pool 中。下次再查这个页里的数据，直接从内存读取，速度起飞。而且，在修改数据时，只需要修改 Buffer Pool 里内存页的数据，脏页会由日志等配合写回磁盘

第二：LRU算法

数据有几百 GB，但 Buffer Pool 只有 8GB，迟早会被塞满。塞满后，新的数据页想进来，就必须踢掉一个旧的页

使用经典的 **LRU（最近最少使用）算法**。搞一个链表，最近被访问的页放在头部，没被访问的页渐渐往尾部掉，满了就把尾部的踢掉

但是，有时候热点数据会被其他数据从LRU链表挤出去，造成缓冲池污染。可以使用像操作系统里面那样的冷热分离机制，只有在冷数据区待到了一定时间并且再次被访问到时，才能够进入到热数据区

第三：B+树还是太慢

B+树虽然依旧很矮胖，但毕竟是要查找3-4次的，InnoDB内部如果发现某个数据页被极其频繁地以同样的条件查询，他会在内存里**自动**为这个页里的数据偷偷建一个 **Hash 表**，下次再查这个数据，直接走 Hash 表，时间复杂度就变为了1